

Woi Audio System

A Lightweight, Production-Ready Audio Framework for Unity

Who Is This For?

You can use this system in two ways:

Basic Mode

- Immediate sounds
- No queue
- No special rules
- Just Play()

Advanced Mode

- Deterministic queue
- Instance limits
- Clip selection logic
- Runtime debugging

You can start simple and grow into advanced features only when needed.

This system does not modify Unity's audio engine.
It only structures and controls AudioSource usage.

1. Introduction

Woi Audio System is a **production-ready audio framework** designed to solve common audio management problems in Unity projects — from small indie games to large-scale productions.

Instead of scattering AudioSource logic across scenes, prefabs, and scripts, this system introduces a **centralized, rule-driven audio architecture** built around:

- Declarative sound behavior (SoundDefinition)
- Explicit scheduling and queue control
- Predictable anti-spam & cooldown rules
- Optional zero-code usage via AudioTrigger
- Runtime visibility through a dedicated Debug Window

The goal is not to replace Unity's audio backend, but to **structure, control, and scale** it.

Key design principles:

- One sound = one definition
- Rules live in data, not scripts
- Debugging audio should be easy
- No hidden magic, no “why did this play twice?” moments

This asset is suitable for **mobile, PC, console, and VR projects**.

What This Asset Is *Not*

- ✗ Not a middleware replacement (FMOD / Wwise)
- ✗ Not an opinionated “music only” solution
- ✗ Not a black box that hides behavior

It is a **clear, extendable Unity-native system** that you fully own.

1.1. System Mental Model

AudioSystem is built on three clear layers:

❶ SoundDefinition → Permanent Behavior

Defines how a sound behaves globally:

- Clip selection mode (Single / Sequence / Random / QueueAll)
- Cooldown
- Delay
- Instance rules (SingleGlobal / Multiple)
- Queue behavior
- 2D / 3D spatial settings

If something is **persistent**, it belongs here.

❷ PlayContext → Temporary Intent

Defines one-shot overrides:

- Volume multiplier
- Pitch multiplier

- Position override
- Follow target
- Explicit clip index
- Ignore cooldowns (debug)

If something is **temporary**, it belongs here.

PlayContext never changes the SoundDefinition.

3 AudioSystem → Runtime Orchestration

Responsible for:

- Applying cooldown
 - Resolving clip selection
 - Handling delays
 - Managing queues
 - Voice limiting
 - Stealing logic
 - Pooling voices
-

Design Philosophy

Persistent behavior → SoundDefinition

Temporary intent → PlayContext

Runtime rules → AudioSystem

This separation prevents logic chaos.

1. ◇ 2) When To Use Queue

Bu bölüm mutlaka olmalı.

2. When Should You Use Queue?

Queue is designed for deterministic, ordered playback.

Good Use Cases

- Narration
- Dialogue sequences
- Tutorial steps
- Music playlists
- Guided voice instructions

Bad Use Cases

- Footsteps
- Gunshots
- UI clicks
- Explosion SFX
- Rapid gameplay sounds

If the order matters → use Queue.

If responsiveness matters → use Immediate.

2. 3) Why Play() Returns Null

Bu support yükünü azaltır.

3. Why Play() May Return null

AudioSystem.Play() may return null intentionally.

This does NOT necessarily mean failure.

It returns null when:

- The sound is routed into the queue
- Cooldown prevents playback
- A delay is scheduled
- Voice limit denies playback

- SingleGlobal ignores retrigger

In these cases:

The system may schedule or defer the sound internally.

If you need guaranteed immediate playback for debugging:

```
audioSystem.Play(sound, PlayContext.DebugNoCooldown());
```

3. ◇ 4) Common Usage Table

Bunu tablo formatında koymalısın.

4. Common Usage Patterns

Scenario	SoundDefinition Setting	How To Call
UI Click	Immediate + Multiple	Play(sound)
Footstep	Immediate + RandomWeighted	Play(sound)
Gunshot	Immediate + Cooldown	Play(sound)
3D Explosion	Immediate + 3D	PlayAt(sound, position)
Dialogue Line	Queue + SingleGlobal	EnqueueAllClips(sound)
Playlist Music	QueueAll	EnqueueAllClips(sound)
Force Play (Debug)	Any	Play(sound, PlayContext.DebugNoCooldown())

4. ◇ 5) PlayContext Clarification (Very Important)

Şunu net yaz:

5. PlayContext – When Overrides Make Sense

PlayContext exists for one purpose:

Temporary, one-shot overrides.

Good Use Cases

- Slight pitch variation
- Position override
- Clip index override
- Debug playback

Bad Use Cases

- Permanent volume rules
- Game design logic
- Global balancing
- Category routing

Persistent behavior must live inside SoundDefinition.

5. ◇ 6) Debug Window Bölümünü Güçlendirelim

6. Debug Window – Shared Space for Designers & Programmers

The Debug Window displays:

- Active sounds
- Queued sounds
- Running / waiting state
- Current queue index
- Skip / Clear controls

It exists to answer one question:

“Why is this sound not playing?”

Use it for:

- QA testing
- Balance validation

- Cooldown debugging
 - Queue validation
 - Voice limit debugging
-

6. ◇ 7) Version Strategy (Doc'a ekle)

En alta küçük bir bölüm:

Versioning Policy

Public API is considered stable starting from version 1.0.0.

Breaking changes to:

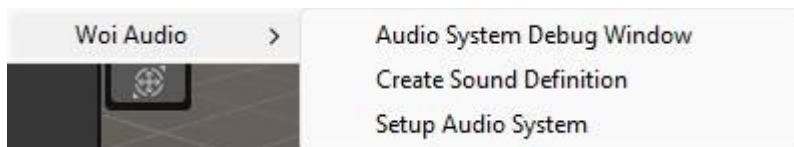
- AudioSystem public methods
- PlayContext structure
- Queue API

Will only occur in major version updates.

2. Quick Start (5 Minutes)

This section gets sound playing **correctly** — not just “making noise”.

You can easily set up with **Tools -> Woi Audio -> Setup Audio System**



Or you can make it manually:

Step 1 — Add AudioSystem to the Scene

1. Create an empty GameObject
2. Add:
 - AudioSystem

- AudioPoolAdapter

Only **one AudioSystem** is required per scene in most projects.

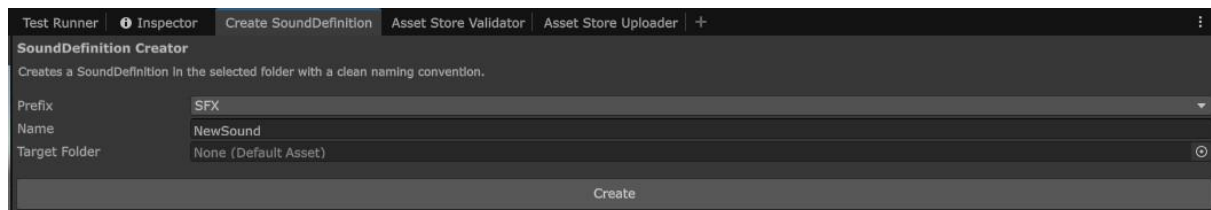
[IMAGE: AudioSystem GameObject with AudioPoolAdapter attached]



Step 2 — Create a SoundDefinition

1. Right-click in Project Window
2. Create → Audio → Sound Definition

Or **Tools -> Woi Audio -> Create Sound Definition**

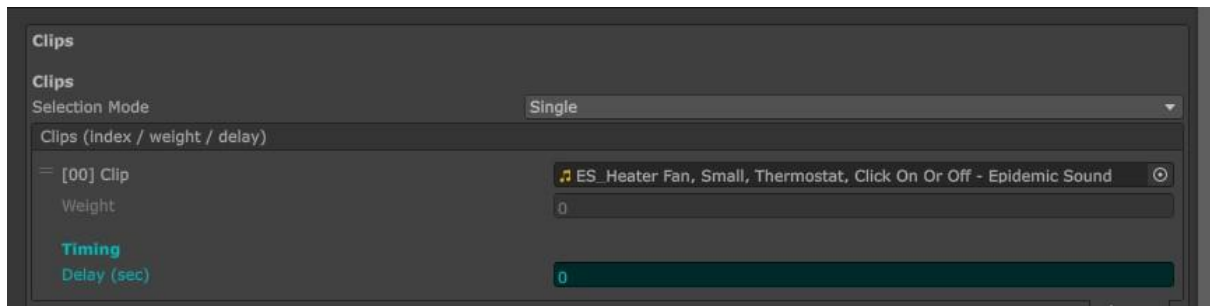


3. Assign one or more AudioClips

This asset defines **everything** about how a sound behaves:

- Clip selection
- Queue rules
- Cooldowns
- Instance limits
- Spatial defaults
- Categories & routing

[IMAGE: SoundDefinition inspector – clips]



Step 3 — Play a Sound (Code)

`AudioSystem.Play(mySound);`

Or with position:

`AudioSystem.PlayAt(mySound, transform.position);`

Or following a transform:

`AudioSystem.PlayFollow(mySound, targetTransform);`

That's it.

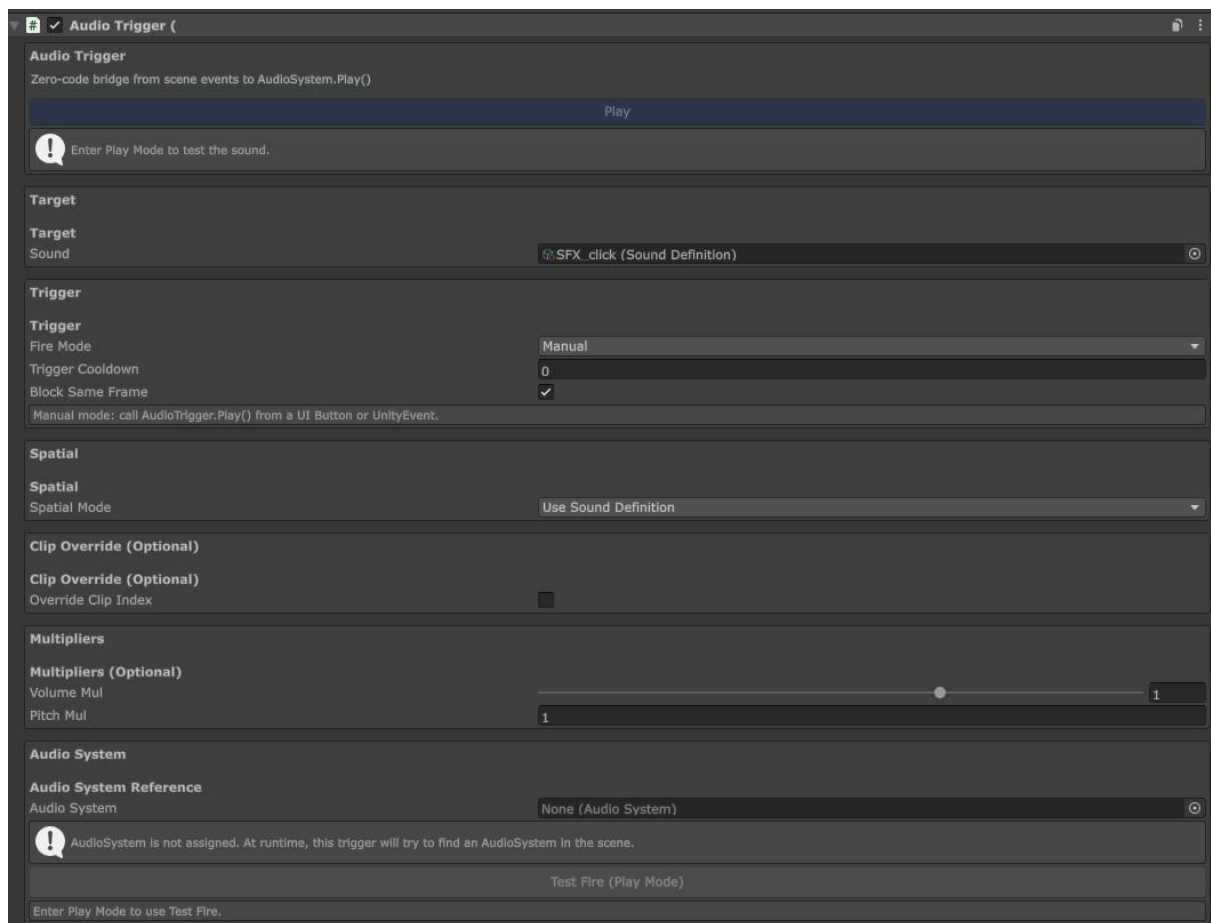
All rules are applied automatically.

Step 4 — Zero-Code Playback with AudioTrigger

If you don't want to write code:

1. Add AudioTrigger to any GameObject
2. Assign a SoundDefinition
3. Choose when it fires (OnEnable, Trigger, Collision, UI Button, etc.)

[Audio Trigger Inspector]



This is fully designer-friendly and production-safe.

3. Core Concepts

Understanding these concepts is key to using the system effectively in **large projects**.

- ☐ **SpatialMode.WorldPosition / FollowTransform**: Context multipliers and clip overrides are preserved (PlayContext is forwarded).
- ☐ **UseSoundDefinition**: No positional override is applied; spatial behavior is controlled by SoundDefinition.spatialBlend etc.

3.1 SoundDefinition (The Heart of the System)

A SoundDefinition is a **data-driven description of a sound**, not just a clip.

It answers questions like:

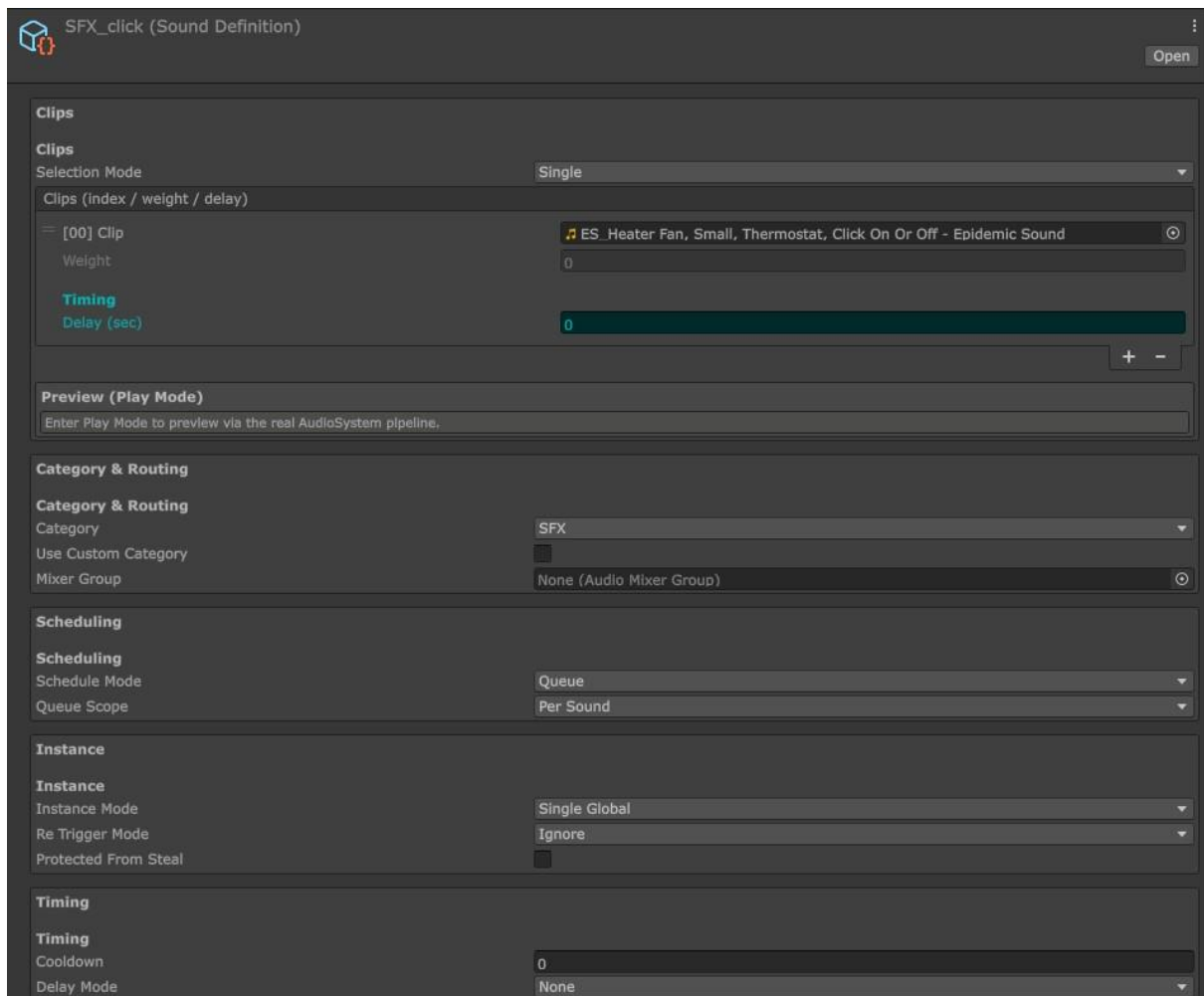
- Can this sound overlap?
- Should it queue?
- Can it spam?

- How is the clip selected? • Is it 2D or 3D by default?
- Does it belong to UI, SFX, VO, Music?

Why this matters:

Changing behavior does **not** require touching code.

[IMAGE: SoundDefinition inspector overview]



3.2 Clip Selection Modes

Each SoundDefinition supports multiple clip strategies:

- **Single**
 - Always play the first clip.
- **Sequence**
 - Plays clips in order, loops back to start.
- **Random Weighted**
 - Weighted randomness with optional no-immediate-repeat.
- **Queue All**
 - Enqueues *all* clips and plays them sequentially.

This is resolved **once per play request**, ensuring predictable behavior.

3.3 Scheduling & Queue System

Sounds can be scheduled in two primary ways:

Immediate

Plays instantly (respecting cooldowns, limits, etc.).

Queue

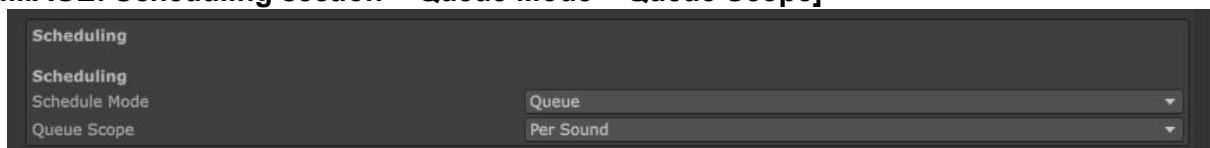
Sounds are queued and played **one after another**, per:

- Sound
- Category

The queue system supports:

- Per-sound or per-category scope
- Duplicate suppression while running
- Manual skipping and clearing
- Runtime inspection (Debug Window)

[IMAGE: Scheduling section – Queue Mode + Queue Scope]



Sequence Mode vs Queue Mode

Although both operate over multiple clips, they serve very different purposes.

Sequence Mode

Sequence is a clip selection strategy.

Every time `Play()` is called:

- The next clip in the list is selected
- Playback happens immediately
- No scheduling occurs
- No automatic progression happens

Example use cases:

- Alternating footstep sounds
- Repeated interaction SFX
- Button toggle sounds
- Weapon fire variation

Sequence does NOT:

- Automatically continue to the next clip
- Manage playback order over time
- Provide Next/Prev navigation
- Create playlists

Queue Mode

Queue is a playback scheduling system.

When a sound is routed into the queue:

- Clips are stored in order
- Playback happens sequentially
- The system waits for each clip to finish
- Next/Prev navigation becomes available
- Autoplay runner manages progression

Example use cases:

- Dialogue sequences
- Narration
- Tutorial instructions
- Music playlists

Key Difference

Sequence decides:

"Which clip should play now?"

Queue decides:

"How should clips play over time?"

3.4 PlayContext (Per-Call Overrides)

Every play request uses a PlayContext.

It allows **per-call overrides** without modifying the SoundDefinition:

- Position / follow target
- Volume & pitch multipliers
- Clip index override
- Cooldown bypass (optional)

This is how the same sound behaves differently in different gameplay contexts.

3.5 Centralized Voice Management

All active sounds are tracked centrally:

- Global max instance limit
- Oldest-first stealing (configurable)
- Optional protection from stealing
- Safe shutdown handling

This avoids:

- Orphaned AudioSources
- Unbounded sound spam
- Hidden performance issues

2. SoundDefinition — Field Reference (Core of the System)

SoundDefinition is the heart of the Woi Audio System.

Almost all playback behavior is defined here — **not in code, not in components**.

This allows designers to fully control audio behavior without touching scripts.

2.1 Clip List

Each SoundDefinition contains one or more **Clip Entries**.

Clip Entry Fields

Clip

The AudioClip to be played.

Weight

Used only when ClipSelectionMode = RandomWeighted.

- Higher weight → higher probability
- If all weights are 0, selection becomes uniform random

Delay (per clip)

Additional delay applied **after the sound-level delay**.

Total Delay = Sound Delay + Clip Delay

Typical use cases

- Foley variations
- Footsteps
- Weapon tails
- UI sound randomization

2.2 Clip Selection

ClipSelectionMode

Single

Always plays the first clip.

Sequence

Plays clips in order (0 → 1 → 2 → ... → 0).

RandomWeighted

Random selection based on clip weights.

QueueAll

Queues **all clips sequentially** and plays them one by one.

QueueAll is ideal for narration, tutorials, or multi-part UI feedback.

noImmediateRepeat

Prevents the same clip from being selected twice in a row

(only applies to RandomWeighted).

Use when

- Random footstep sounds
- Ambient variations
- UI click sounds

2.3 Scheduling

ScheduleMode

Immediate

Sound is played instantly (subject to cooldown, instance rules, etc.).

Queue

Sound is routed through the internal queue system.

Queue mode guarantees:

- Ordered playback
- No overlap
- Deterministic behavior

QueueScope

Defines how queue instances are grouped.

PerSound

Each SoundDefinition has its own queue.

PerCategory

All sounds sharing the same category use a single queue.

Typical example

- UI sounds → PerCategory
- Narration → PerSound suppressDuplicatesWhileQueued When enabled:
- If a queue is already running
- The same sound (or same clip signature) will **not be enqueued again**

This prevents:

- Button spam
- Repeated narration triggers
- UI noise overload

This is a **runtime-safe deduplication**, not a cooldown.

2.4 Timing

Cooldown

Minimum time (in seconds) between successful plays of this sound.

- Works for both Immediate and Queue
- Can be bypassed via `PlayContext.ignoreCooldowns`

Use when

- Weapon fire sounds
- UI clicks
- Damage sounds

DelayMode

Fixed

Always uses the same delay.

RandomRange

Random delay between `delayRange.x` and `delayRange.y`.

Useful for:

- Natural variation
- Avoiding robotic timing

2.5 Instance Rules

InstanceMode

Multiple

Unlimited instances (subject to global max voices).

SingleGlobal

Only one instance may exist at a time.

If retriggered:

- Behavior depends on `ReTriggerMode`

`ReTriggerMode` (SingleGlobal only)

Restart

Stops the existing sound and plays again.

Ignore

Keeps the existing sound, ignores the new request.

protectedFromSteal

If enabled:

- This sound cannot be force-stopped when max voices are reached

Use for:

- Critical narration
- Important feedback sounds

2.6 Playback Controls

Volume & Pitch

Base volume and pitch for the sound.

These values are later multiplied by:

- PlayContext.volumeMul
- PlayContext.pitchMul This allows:
 - Per-trigger scaling
 - Distance-based logic
 - Dynamic feedback

Loop

If enabled:

- The sound loops
- Looping sounds **do not block queues**

Common use:

- Ambient loops
- Machine hums
- Environmental sounds

2.7 3D Audio

Spatial Blend

- 0 → Fully 2D
- 1 → Fully 3D 3D sounds support:

- Min / Max distance
- Rolloff curves
- Follow targets

2.8 Categories

Each SoundDefinition belongs to a **Category**.

Categories are:

- ScriptableObject-based
- User-extensible
- Used for grouping, queuing, and debugging

Typical categories:

- UI
- SFX
- Music
- Narration
- Ambient

Summary

SoundDefinition provides:

- Deterministic audio behavior
 - Designer-first workflow
 - Zero-code iteration
 - Production-ready control
- If you understand **SoundDefinition**, you understand

the entire system.

3. AudioTrigger — Zero-Code Playback Component AudioTrigger is a **designer-first MonoBehaviour** that allows sounds to be played without writing any code.

It acts as a bridge between:

- Scene events (Enable, Trigger, Collision, UI)
- AudioSystem.Play(...)
- SoundDefinition rules

All complex logic (queueing, cooldowns, instance rules) remains centralized inside the audio system.

3.1 Purpose & Philosophy

AudioTrigger exists to solve three problems:

7. **No scripting for common audio use cases**
2. **No duplicated logic across scenes**
3. **Consistent behavior across all triggers**

AudioTrigger never decides *how* a sound behaves —

it only decides *when* the sound is requested.

3.2 Fire Modes

FireMode

Defines **when** the trigger fires.

ModeDescriptionManualTriggered manually via UnityEvent or ButtonOnEnableFires when GameObject becomes activeOnDisableFires when GameObject is disabledOnTriggerEnterRequires Collider (IsTrigger)OnTriggerExitRequires Collider (IsTrigger)OnCollisionEnterRequires Rigidbody + ColliderOnCollisionExitRequires Rigidbody + Collider

Typical usage

- UI buttons → Manual
- Spawn SFX → OnEnable
- Pickup sound → OnTriggerEnter
- Destruction sound → OnDisable

3.3 Trigger-Level Anti-Spam triggerCooldown

A **local cooldown**, independent from SoundDefinition.cooldown.

- Prevents the same trigger from firing repeatedly
- Does NOT affect other triggers or direct Play calls

Use cases:

- Rapid trigger enter/exit
- Physics jitter
- UI hover spam blockSameFrame

If enabled:

- Prevents multiple fires within the same frame

This is critical for:

- UI buttons
- Physics callbacks
- Multi-event chains

Allows the trigger to force a specific clip index from the SoundDefinition.

Important

- Does NOT bypass queue or cooldown rules
- Only overrides clip selection

Use cases:

- UI buttons sharing one SoundDefinition
- Specific feedback sounds from a shared pool

3.5 Spatial Playback Control

SpatialMode

Controls **where** the sound is played.

ModeBehaviorUseSoundDefinitionUses SoundDefinition spatial

settingsWorldPositionPlays at a fixed world positionFollowTransformFollows a transform
(3D sound) positionSource

Used only in WorldPosition mode.

If null:

- Falls back to the AudioTrigger's transform position followTarget

Used only in FollowTransform mode.

If null:

- Falls back to the AudioTrigger's transform

3.6 Multipliers (Dynamic Scaling)

volumeMul

Multiplies SoundDefinition volume at runtime.

- Default: 1.0
- Recommended range: 0 → 1.5

Used for:

- UI emphasis
- Distance scaling
- Contextual feedback pitchMul

Multiplies SoundDefinition pitch at runtime.

Used for:

- Variation
- Gameplay feedback
- Speed-based effects 3.7 AudioSystem Resolution audioSystem (optional)

If not assigned:

- AudioTrigger automatically finds the first AudioSystem in the scene

This ensures:

- Zero setup friction
- Safe defaults
- Works with DontDestroyOnLoad systems

3.8 Events

onPlayed

Invoked after a successful Play request.

Useful for:

- Chaining logic
- Analytics
- Debug hooks onBlocked

Invoked when the trigger was blocked due to:

- Cooldown
- Same-frame block
- Missing references
- Shutdown state

3.9 Runtime Flow (Conceptual)

1. Trigger fires
2. Local spam checks
3. PlayContext is built
4. AudioSystem.Play(sound, context)
5. SoundDefinition rules apply
6. AudioVoice created (or queued)

AudioTrigger never bypasses system rules.

Summary

AudioTrigger provides:

- Zero-code audio playback
- Safe defaults

- Strong anti-spam protection
- Clear separation of responsibility

Designers work with **AudioTrigger**

Programmers work with **SoundDefinition & AudioSystem**

4. Queue System — Deterministic Audio Flow

The Queue System is one of the **core differentiators** of this asset.

Unlike simple “play-after-delay” solutions, this system provides:

- Deterministic playback order
- Category- or sound-scoped queues
- Optional duplicate suppression
- Runtime visibility and control

4.1 Why a Queue System?

Typical audio systems fail in these scenarios:

- UI spam causes overlapping clicks
- Narration lines overlap or interrupt each other
- Sequential SFX lose order under load
- Designers rely on manual delays

This Queue System solves all of the above **without manual timing**.

4.2 Queue Scope

Each SoundDefinition defines **where it queues**.

QueueScope

ScopeBehaviorPerSoundOnly this sound queues with itselfPerCategoryAll sounds in the same category share one queue

Examples

- UI clicks → PerCategory
- Narration lines → PerCategory
- Weapon reload → PerSound

4.3 Schedule Mode

ScheduleMode.Queue

When enabled:

- Play requests are **enqueued instead of played immediately**
- Queue execution guarantees order

ScheduleMode.Immediate

- Sound plays immediately
- Still respects instance rules and cooldowns

Queueing is **opt-in**, not global.

4.4 Clip Selection & Queue Interaction

ClipSelectionMode.QueueAll

Special mode that enqueues **all clips** in the SoundDefinition sequentially.

Typical use cases:

- Narration lines
- Tutorial VO
- Multi-step audio feedback

Behavior:

1. All clips are enqueued with explicit clip indices
2. Queue plays them one-by-one
3. Each clip respects its own delay

4.5 Duplicate Suppression (Anti-Spam)

suppressDuplicatesWhileQueued

When enabled:

- If the same sound (or clip) is already queued or playing
- New requests are ignored

This is **not a cooldown**.

Key differences:

- Only active while the queue is running
- Resets automatically when the sound finishes

Practical Examples

ScenarioSettingUI button spamEnabledRapid-fire weaponDisabledNarration
VOEnabledEnvironmental loopsDisabled

4.6 Queue Execution Model

Internally:

- Each queue has a single coroutine runner
- Only one sound plays per queue at a time
- Looping sounds do **not block** the queue

Execution steps:

1. Resolve clip
2. Resolve delays
3. Play sound
4. Wait until completion
5. Move to next item

4.7 Runtime Control API

The AudioSystem exposes full queue control:

```
audioSystem.GetQueueCount(sound);
```

```
audioSystem.SkipQueueOne(sound);
```

```
audioSystem.ClearQueue(sound);
```

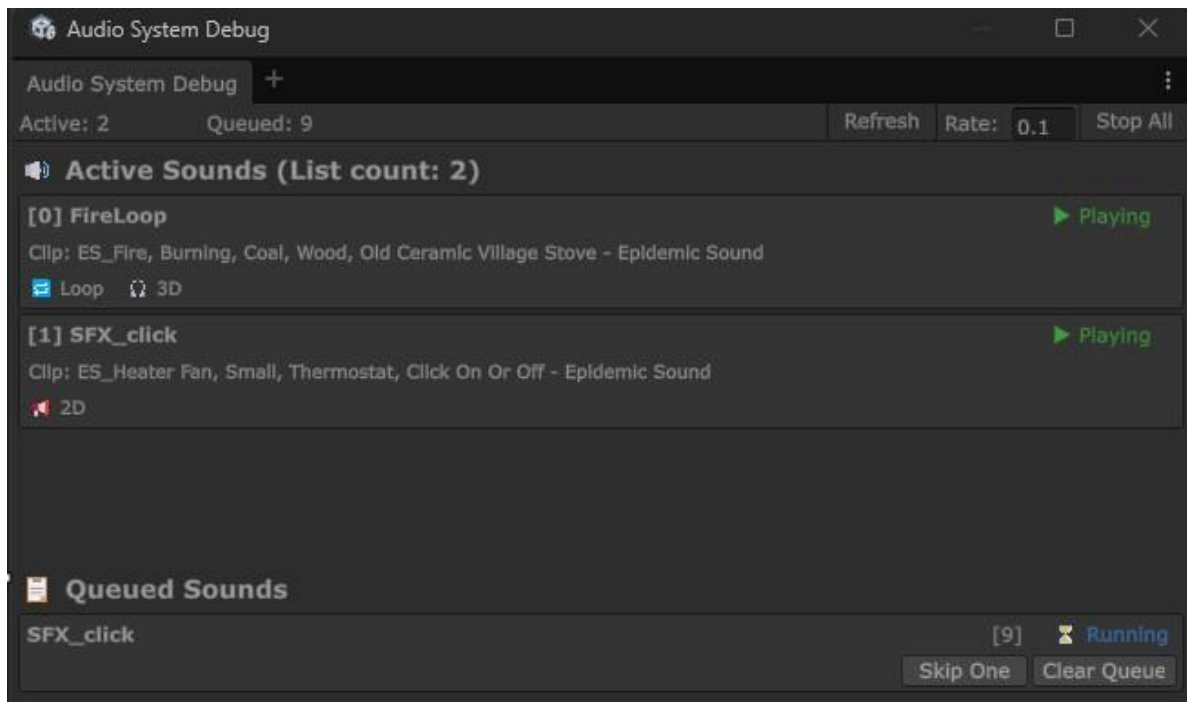
```
audioSystem.ClearQueue();
```

This enables:

- Debug tools
- Cutscene skipping
- Emergency cleanup

4.8 Queue Debug Visibility

The queue system is fully observable via:



Audio System Debug Window

- Active sounds
- Queued sounds
- Running vs waiting queues
- Skip / Clear controls

This is a **major UX advantage** over black-box audio systems.

Summary

The Queue System provides:

- Predictable audio flow
- Designer-safe sequencing
- Strong anti-spam without cooldown abuse
- Runtime visibility & control

This is suitable for:

- UI-heavy games
- Narrative-driven games
- Casual & hypercasual titles
- Large-scale projects with many audio triggers

5. Instance Management & Voice Limits

This system uses **explicit instance management** instead of relying on

AudioSource defaults or Unity's implicit limits.

The goal is:

- Predictable behavior under load
- No runaway AudioSources
- Designer-controlled prioritization

5.1 Why Explicit Instance Control?

Default Unity behavior:

- Unlimited AudioSources can play
- No global limit
- No prioritization logic
- Hard to debug under stress

This asset introduces a **centralized voice registry** that tracks every active sound.

5.2 Global Voice Limit

AudioSystemConfig → maxSoundInstances

Defines the **maximum number of concurrent active sounds**.

When the limit is reached:

1. A steal candidate is searched
2. The oldest non-protected voice is stopped
3. New sound is allowed to play

If no candidate is found:

- The new sound is silently dropped

This ensures:

- Stable performance
- No audio overload
- No frame spikes

5.3 Voice Registry

Internally, the system maintains:

- An ordered list of active voices (oldest → newest)
- A fast lookup map for registration

This allows:

- O(1) add/remove
- Deterministic steal order
- Accurate debug visibility

5.4 Instance Modes

Each SoundDefinition defines how instances behave.

InstanceMode

ModeBehaviorMultipleUnlimited instances (within global limit)SingleGlobalOnly one instance allowed

SingleGlobal Behavior

If a sound is marked SingleGlobal:

- New play request arrives
- Existing instance is checked

Then:

- **Ignore** → new request is ignored
- **Restart** → existing sound is stopped and restarted

This is ideal for:

- Ambient loops
- Music
- Persistent UI sounds

5.5 Protected Voices protectedFromSteal

When enabled:

- This sound will **never** be force-stopped by the global limit

Typical use:

- Music
- Critical feedback
- Long narration lines

Protected sounds can still be stopped manually.

5.6 ReTrigger Mode

Controls behavior when a sound is already playing.

ModeResultIgnoreKeep current instanceRestartStop & replay

Applies only to SingleGlobal sounds.

5.7 Lifecycle Management

Each AudioVoice follows a strict lifecycle:

1. Pulled from pool
2. Registered in AudioSystem
3. Played
4. Stopped or finished
5. Unregistered
6. Returned to pool

This ensures:

- No leaked AudioSources
 - Accurate active counts
 - Clean shutdown behavior
- ### 5.8 Shutdown Safety During:
- Scene unload
 - Application quit
 - Play mode exit

The system:

- Stops all active voices
- Clears queues
- Prevents new plays

This avoids Unity teardown errors and editor crashes.

Summary

Instance Management guarantees:

- Stable audio under pressure
- Clear ownership of sounds
- No uncontrolled overlap
- Safe behavior in large projects

This makes the system suitable not only for casual games,

but also **production-scale projects**.

6. Clip Selection & Randomization

The Clip Selection system defines **which AudioClip is chosen** when a sound is played.

This logic is fully deterministic, debuggable, and independent from playback timing.

All selection rules live inside SoundDefinition.

6.1 Why Explicit Clip Selection?

Common problems in audio systems:

- Random clips repeating immediately
- Sequence state lost across scenes
- Queue + random behaving unpredictably
- Hard-coded logic scattered across scripts

This asset solves those by:

- Centralizing clip selection
- Persisting state per SoundDefinition
- Making selection mode explicit and visible

6.2 ClipSelectionMode Overview

ClipSelectionMode

ModeBehaviorSingleAlways plays the first clipSequencePlays clips in order,

loopingRandomWeightedWeighted random selectionQueueAllEnqueues all clips sequentially

6.3 Single

The simplest mode.

Behavior:

- Always plays clips[0]
- Ignores weights and randomness

Use cases:

- Music
- Ambient loops
- One-off sounds

6.4 Sequence

Plays clips **in a fixed order**, looping back to the start.

Behavior:

1. Start at index 0
2. Increment after each play
3. Wrap when reaching the end
4. State is stored per SoundDefinition

Important:

- Sequence state is **shared across all triggers**
 - Queue respects sequence order

Use cases:

- Footsteps
- UI navigation sounds
- Pattern-based SFX

6.5 RandomWeighted

Selects a clip based on per-clip weights.

Weight System

- Each clip has a weight
- Total weight is summed
- Random value selects a range

If all weights are zero:

- Falls back to uniform random

No Immediate Repeat

Optional toggle:

- Prevents the same clip from playing twice in a row

Implementation details:

- Stores last played index
- Retries selection a few times
- Forces a different clip if needed

Use cases:

- Repetitive actions
- UI sounds
- Combat feedback

6.6 QueueAll (Special Mode)

QueueAll is **not just a selection mode** — it changes scheduling behavior.

Behavior:

- All clips are enqueued immediately
- Each clip is played in order

- Each clip uses its own delay

Important:

- QueueAll ignores random/sequence logic
- Clip indices are explicit
- Fully deterministic Typical use cases:
 - Narration lines
 - Tutorial VO
 - Audio instructions

6.7 Clip Override via PlayContext

Any system (AudioTrigger, code, debug tools) can override clip

selection: `ctx = ctx.SetClipIndex(index);` Rules:

- Overrides selection mode
- Still respects queue, cooldown, instance rules
- Used internally by QueueAll

6.8 Interaction with Queue System

Selection ModeQueue InteractionSingleQueues as one itemSequenceQueues each play in orderRandomWeightedResolved at play timeQueueAllEnqueues all clips immediately

This ensures:

- No surprises
- No hidden randomness
- Full predictability

Summary

Clip Selection provides:

- Explicit behavior
- Zero hidden randomness
- Designer-friendly control
- Safe interaction with queues

This system is robust enough for:

- Casual games
- Narrative-heavy projects
- Long-running sessions
- Debug-heavy production workflows

7. Debug Tools & Runtime Visibility

One of the strongest differentiators of this asset is that **nothing is a black box**.

Every active sound and every queued request can be inspected **live** in the Editor.

This dramatically improves:

- Debugging speed
- Designer confidence
- Production safety

7.1 Audio System Debug Window

The **Audio System Debug Window** is a dedicated Editor tool that displays:

- Active playing sounds
- Queued sounds per SoundDefinition
- Queue execution state
- Runtime controls (skip / clear)

It works **only in Play Mode** and updates automatically at runtime.

7.2 Active Sounds Panel

Information Shown

For each active sound:

- Sound name
- Clip name
- Playing / stopped state
- Looping flag
- 2D / 3D spatial mode
- Follow target presence Visual indicators:
 - ► Playing (green)
 - ■ Stopped (red)
- 2D / 3D icons
- Loop / Follow icons

This makes it immediately clear:

- What is playing
- Why it is playing
- How it is spatialized

7.3 Queued Sounds Panel For each queued sound:

- SoundDefinition name • Number of queued items • Queue state:
 - Running
 - Waiting

Controls:

- **Skip One** → removes the next item
- **Clear Queue** → removes all items for that sound

This is extremely useful for:

- Narration skipping • UI flow debugging
- Stress testing queue behavior

7.4 Multi-AudioSystem Support

If multiple AudioSystems exist in the scene:

- The debug window displays a dropdown selector
- Each system can be inspected independently

This supports:

- Additive scenes
- Multi-world setups
- DontDestroyOnLoad audio managers

7.5 Refresh Strategy

The window:

- Refreshes automatically at a configurable rate
 - Uses Editor-time ticking
- Key characteristics:
- No frame dependency
 - No user interaction required
 - Safe in Editor focus loss
 - Lightweight polling

7.6 Runtime Snapshot API

Internally, the AudioSystem exposes snapshot methods:

- `GetActiveVoicesSnapshot()`
- `GetQueuedSoundsSnapshot()`

These:

- Return immutable data
 - Never expose live references
 - Are safe for Editor tooling
- This separation ensures:
- Zero runtime side effects
 - No accidental state mutation
 - Clean debug architecture

7.7 Why This Matters

Most audio systems fail when something goes wrong because:

- You can't see what's happening
- You can't tell why something didn't play
- You can't inspect queues

This asset solves that **by design**.

Debug visibility is treated as a **first-class feature**, not an afterthought.

Summary

Debug Tools provide:

- Full runtime transparency
- Instant feedback
- Safe control over queues
- Confidence in large projects

This significantly reduces:

- Audio bugs
- Iteration time
- Designer frustration

8. System Architecture & Extensibility

This audio system is designed with a **layered, responsibility-separated architecture**.

Each part has a single job, which makes the system:

- Predictable
- Extensible
- Safe to modify
- Easy to document

8.1 High-Level Architecture Overview

At runtime, the flow looks like this:

AudioTrigger / Code → **AudioSystem** → **QueueController** → **AudioVoice** → **AudioSource**

Each layer answers a specific question:

LayerResponsibilityAudioTrigger*When* should a sound playAudioSystem*Whether* it should playQueueController*When in order* it should playAudioVoice*How* it playsAudioSourceUnity playback

No layer reaches across responsibilities.

8.2 AudioSystem (Core Brain)

AudioSystem is the **single source of truth**.

It owns:

- Cooldowns
- Instance limits • Queue routing
- SingleGlobal logic
- Voice lifecycle
- Debug snapshots

Key principles:

- Stateless input (PlayContext)
- Centralized decision making
- No UnityEvent dependencies
- No editor-only logic This makes it:
- Safe in runtime builds
- Deterministic
- Easy to unit-test

8.3 QueueController (Deterministic Scheduling)

QueueController is fully isolated from playback logic.

It:

- Owns queues
- Owns runners
- Owns duplicate suppression
- Owns queue lifetime

Important design decisions:

- Queue keys are deterministic
- Per-sound vs per-category behavior is explicit
- Queue runners are self-cleaning
- No AudioSource access

This allows:

- UI click queues
- Narration chains
- Cutscene sequencing
- Stress-safe spam protection

8.4 AudioVoice (Playback Unit) AudioVoice

represents **one playback instance**.

It:

- Wraps a pooled AudioSource
- Applies SoundDefinition + PlayContext
- Knows if it is playing
- Reports lifecycle events back to AudioSystem

Why this matters:

- No AudioSource leaks
- Pooling is invisible to designers
- Runtime limits are enforced centrally
- Stealing logic is safe

8.5 SoundDefinition (Behavior Contract)

SoundDefinition is **not just data** — it is a behavior contract.

It defines:

- Clip selection logic
- Queue behavior
- Cooldowns
- Instance rules
- Looping
- Stealing protection
- Spatial defaults

Designers never touch code.

Programmers never hardcode audio rules.

8.6 PlayContext (Immutable Intent)

PlayContext is a **one-shot intent container**.

It allows:

- Position override
- Follow override
- Clip index override
- Volume / pitch multipliers
- Cooldown bypass
- Queue flags

Key rule:

SoundDefinition defines policy, PlayContext defines intent

This separation prevents:

- Hidden side effects
- Hardcoded behavior
- Fragile special cases

8.7 Editor-Only Systems Are Isolated

Editor tools:

- Debug Window
- Inspectors
- Validation warnings Are:
- Fully Editor-only
- Snapshot-based
- Zero runtime impact

No `#if UNITY_EDITOR` leaks into runtime logic.

8.8 Extending the System

Common extension points:

Add a new trigger type

→ Extend `AudioTrigger.FireMode`

Add a new scheduling rule

→ Extend `QueueController`

Add new debug data

→ Extend snapshot structs only

Add new selection logic

→ Extend `ClipSelectionMode` + resolver

No refactoring required.

8.9 Why This Architecture Scales

This system has already proven viable for:

- UI-heavy mobile games

- VR training simulations
- Narrative-driven flows
- Large multi-scene projects Because:
- No global static audio state • No hidden implicit behavior
- No editor-runtime coupling

Summary

Architecture strengths:

- Clear separation of concerns
- Deterministic behavior
- Safe extensibility
- Production-ready structure

This is not a “small helper script”.

It is a **proper audio framework**, intentionally kept lightweight.

9. UX & Designer Workflow

One of the main goals of this system is the following:

Designers define how a sound behaves — not how it is played.

Code only sends a *Play intent*.

9.1 SoundDefinition Inspector – How to Think About It

Each field in SoundDefinition answers **exactly one question**.

Because of this, designers do **not** need to memorize every toggle individually.

Use the following mental model:

- ◇ 1 “WHEN should this sound play?”

Schedule Mode

- **Immediate** → Play instantly
- **Queue** → Play in sequence

☞ UI clicks, narration, tutorials → **Queue**

☞ Gunshots, explosions → **Immediate**

- ◇ 2 “HOW MANY instances can exist at the same time?”

Instance Mode

- **Unlimited** → Regular SFX
- **SingleGlobal** → Music, ambience, narrator

ReTrigger Mode (SingleGlobal only)

- **Ignore** → Ignore retriggers
- **Restart** → Restart playback

◇ 3 "What happens if this sound is spammed?"

Cooldown

Sound-level anti-spam protection.

Suppress Duplicates While Queued

- Enabled for UI clicks and menu sounds
- Disabled for rapid-fire or machine-gun sounds

This toggle is specifically designed to solve **UI click spam** issues.

◇ 4 "If there are multiple clips, how should they be selected?"

Clip Selection Mode

- **Single** → One clip only
- **Sequence** → Play clips in order
- **Random Weighted** → Weighted random selection
- **Queue All** → Play all clips sequentially

No Immediate Repeat

- Only meaningful for Random selection

◇ 5 "Is this sound 2D or 3D?"

Spatial Blend

- 0 → 2D
- 1 → 3D

Min / Max Distance

- Only used for 3D sounds

The spatial decision is made in **SoundDefinition**.

Overrides at play time are optional.

9.2 AudioTrigger – Zero-Code Workflow

AudioTrigger solves the following problem:

“Let designers trigger sounds without writing code.”

When to Use

- UI buttons
- Trigger volumes
- Collision feedback
- Enable / Disable state sounds

When NOT to Use

- Complex gameplay logic
- Network-authoritative audio events

Fire Mode Logic

- **Manual** → UI Button, UnityEvent
- **OnEnable / OnDisable** → State-based SFX
- **Trigger / Collision** → Physical feedback

Trigger Cooldown vs Sound Cooldown

TypePurposeTrigger CooldownPrevents spam from the same componentSound CooldownGlobal sound-level spam protection These

are **intentionally separate**.

8. 9.3 PlayContext – When Overrides Make

Sense

PlayContext exists for one purpose:

Temporary, one-shot overrides at call time.

It is not a configuration system.

It is not a behavior system.

It is not a rule system.

It is simply a way to say:

“For this specific playback call, do it slightly differently.”

What PlayContext Is

PlayContext modifies a single playback request.

It can temporarily override:

- Volume multiplier
- Pitch multiplier
- World position
- Follow target
- Clip index
- Cooldown bypass (debug use)

Once the call is done, the override is gone.

No state is stored.

No rules are changed.

No SoundDefinition is modified.

Good Use Cases

Use PlayContext when you need small, runtime variation:

- Volume / pitch variation
 - Slight randomization
 - Dynamic mix adjustments
 - Context-based loudness changes
- Position override
 - Play at a hit point
 - Override emitter position
- Follow override

- Attach sound to a moving object
 - Clip index override
 - Cinematic sequencing
 - Debugging a specific variant
 - Deterministic playback
 - Debug bypass
 - Temporarily ignore cooldown rules
-

Bad Use Cases

Do NOT use PlayContext for:

- Permanent behavior rules
- Gameplay logic
- Sound system architecture decisions
- “This sound should always behave like X”

If behavior must be consistent and persistent,
it belongs inside **SoundDefinition**.

Clear Rule

Persistent behavior = **SoundDefinition**

Temporary intent = **PlayContext**

If you are writing logic that always applies →
SoundDefinition.

If you are modifying a single playback call →
PlayContext.

9.4 Debug Window – Shared Space for Designers & Programmers

The Debug Window shows:

- Active sounds
- Queued sounds
- Running / waiting states
- Skip / Clear controls

This window is critical for:

- QA
- Balancing
- Answering “Why is this sound not playing?”

10. Best Practices

☒ UI Sounds

- Queue
- SuppressDuplicatesWhileQueued = ON
- Short cooldown (0.05–0.1)

☒ Weapons / Rapid Fire

- Immediate
- Duplicate suppression OFF
- No queue

☒ Narration / Tutorials

- Queue
- PerCategory
- Loop = OFF

☒ Music / Ambience

- SingleGlobal
- Loop = ON
- ProtectedFromSteal = ON

11. Common Mistakes (and Why)

✗ Queue + Loop

→ The queue never finishes

✗ Random + NoImmediateRepeat with a single clip

→ Meaningless configuration

✗ High Cooldown + High TriggerCooldown together

→ Sound appears “broken”

✗ 3D sound + Play() without position

→ Sound behaves like 2D

12. Performance Notes

- AudioSource pooling is used
- No runtime allocations
- GC-free
- Debug system is snapshot-based
- Editor-only

This system is safe for:

- Mobile
- VR
- Large scenes

13. FAQ (Short)

Q: Can I have multiple AudioSystems in a scene?

A: Yes. The Debug Window supports multiple systems.

Q: Is it network-safe?

A: Playback is local. Events can come from the network.

Q: Is it DOTS / ECS compatible?

A: Playback uses classic AudioSource, but ECS-friendly calls are possible.

14. When NOT to Use This System

- Music-only projects
- Very small prototypes
- One-shot game jam scripts

Final Notes

This asset is:

- Minimal
- Deterministic
- Production-safe
- Designer-friendly But:
- Not overengineered
- Not a black box

10.1 Playing a Specific Clip (Example: 20th Clip)

If a SoundDefinition contains multiple clips and you want to play a specific one (for example the 20th clip):

Note: Clip index is zero-based.
The 20th clip = index **19**.

Basic Example

```
AudioSystem.Play(sound, PlayContext.WithClipIndex(19));
```

This overrides the clip selection mode and forces that specific clip to play.

With Position Override (3D Playback)

```
var ctx = PlayContext.Default  
    .SetClipIndex(19);
```

```
audioSystem.PlayAt(sound, hitPoint, ctx);
```

This:

- Forces clip index 19
- Plays it at a world-space position

With Follow Target

```
var ctx = PlayContext.WithClipIndex(19);  
audioSystem.PlayFollow(sound, targetTransform, ctx);
```

10.2 Volume & Pitch Variation

```
var ctx = PlayContext.Default  
    .SetVolumePitch(0.9f, 1.1f);
```

```
audioSystem.Play(footstepSound, ctx);
```

Use this for:

- Subtle randomness
- Context-based mixing
- Dynamic audio feedback

10.3 Debug Playback (Ignore Cooldown)

For debugging purposes only:

```
audioSystem.Play(sound, PlayContext.DebugNoCooldown());
```

This bypasses:

- Cooldowns
- Delays
- Queue routing rules

Use carefully — not recommended for gameplay logic.

10.4 Queue / Playlist Example

Enqueue all clips and navigate:

```
audioSystem.EnqueueAllClips(musicPack);
audioSystem.QueueNext(musicPack);
```

10.5 Understanding Play() Returning null

Play() may return null when:

- The sound is routed into the queue
- Cooldown prevents playback
- A delay is scheduled
- Instance limits deny playback

This does **not** necessarily mean failure — it may mean the sound is scheduled or deferred.

11. Optimization & Runtime Safety

Woi Audio System is designed not only for clarity, but also for performance and production safety.

This section explains how the system remains stable under load.

11.1 Zero Runtime Allocations During Playback

During normal playback:

- No LINQ is used
- No reflection is used
- No dynamic GameObject creation occurs
- No string allocations occur
- No hidden closures are created

All playback operations use:

- Preallocated collections
- Object pooling (AudioVoice)
- Deterministic dictionaries
- Struct-based PlayContext

This ensures:

- No GC spikes during rapid SFX playback
 - Safe behavior under stress (e.g. 100+ rapid calls)
 - Stable performance in VR and mobile
-

11.2 AudioSource Pooling

Each AudioVoice wraps a pooled AudioSource.

Lifecycle:

Pulled from pool

Registered

Played

Stopped / finished

Returned to pool

No AudioSource is created at runtime during playback.

Benefits:

- No hidden instantiation cost
 - No memory fragmentation
 - Predictable performance
 - Safe long play sessions
-

11.3 Global Voice Limit Protection

AudioSystemConfig → maxSoundInstances

If the limit is reached:

- The oldest non-protected voice is stopped
- Or the new sound is denied (if no candidate exists)

This prevents:

- Audio overload
- Frame drops due to excessive voices
- Unbounded sound spam

Voice stealing is:

- Deterministic
 - Centralized
 - Fully visible in Debug Window
-

11.4 Safe Shutdown Handling

During:

- Scene unload
- Application quit
- Play mode exit
- OnApplicationPause

The system:

- Stops all active voices
- Clears all queues
- Prevents new playback
- Avoids Unity teardown exceptions

This avoids:

- "AudioSource destroyed but still playing" errors
 - Editor crashes
 - Invalid coroutine states
 - Ghost AudioSources in hierarchy
-

11.5 Queue Runner Safety

Each queue:

- Uses a single coroutine runner
- Self-terminates when empty
- Cleans itself after completion

Protection mechanisms include:

- Timeout fallback
- Grace period before playback detection
- Loop-safe behavior
- Null checks at every step

This ensures:

- No stuck queues
 - No infinite loops
 - No dead coroutines
 - No silent lockups
-

11.6 Deterministic Behavior Under Stress

Even under heavy load:

- Cooldowns are respected
- Duplicate suppression works
- Instance limits are enforced
- Queue order is preserved

There is no hidden race condition between:

- Clip selection
- Queue scheduling
- Instance management

All decisions flow through AudioSystem.

11.7 Editor Debug Is Isolated

The Debug Window:

- Uses snapshot structs
- Never exposes live references
- Does not mutate runtime state
- Exists only in the Editor

There is no debug overhead in builds.

11.8 Suitable For Production

This system is safe for:

- Mobile projects
- VR training simulations
- Narrative-heavy games
- UI-intensive projects
- Long-running sessions
- Multi-scene setups

It was intentionally designed to:

- Avoid black-box behavior
- Avoid magic auto-fixes
- Avoid hidden runtime allocations
- Avoid uncontrolled AudioSource creation

Safety Guarantees

Woi Audio System guarantees:

- No uncontrolled AudioSource creation
- No hidden static state corruption
- No cross-scene memory leaks
- No unbounded queue growth
- No runtime editor coupling
- No silent audio spam under load

If something does not play, there is always a reason — and it is visible.

Why This System Exists

Most Unity audio setups break under scale.

This system exists to provide:

Deterministic behavior

Clear ownership of sound logic

Safe queue scheduling

Production-ready instance control

Full runtime visibility

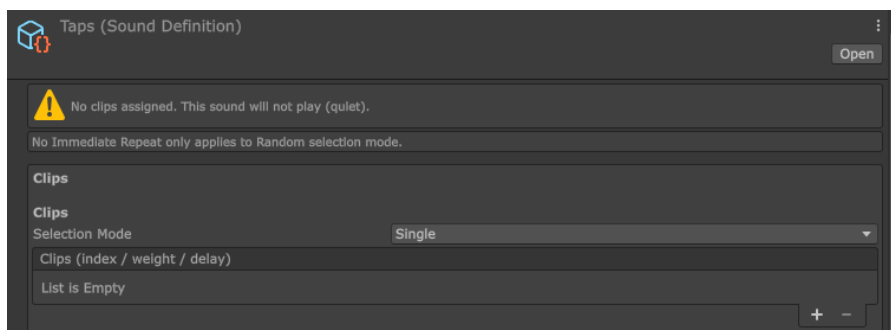
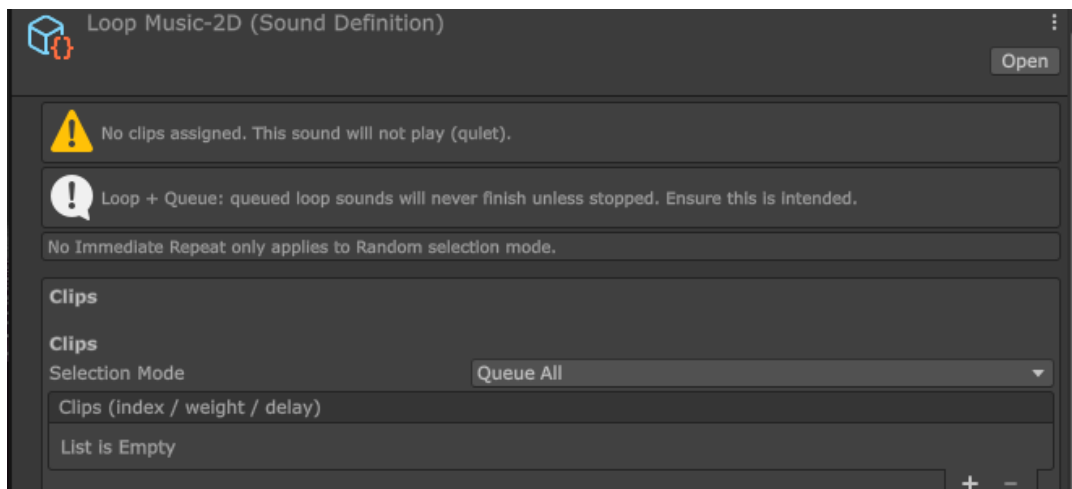
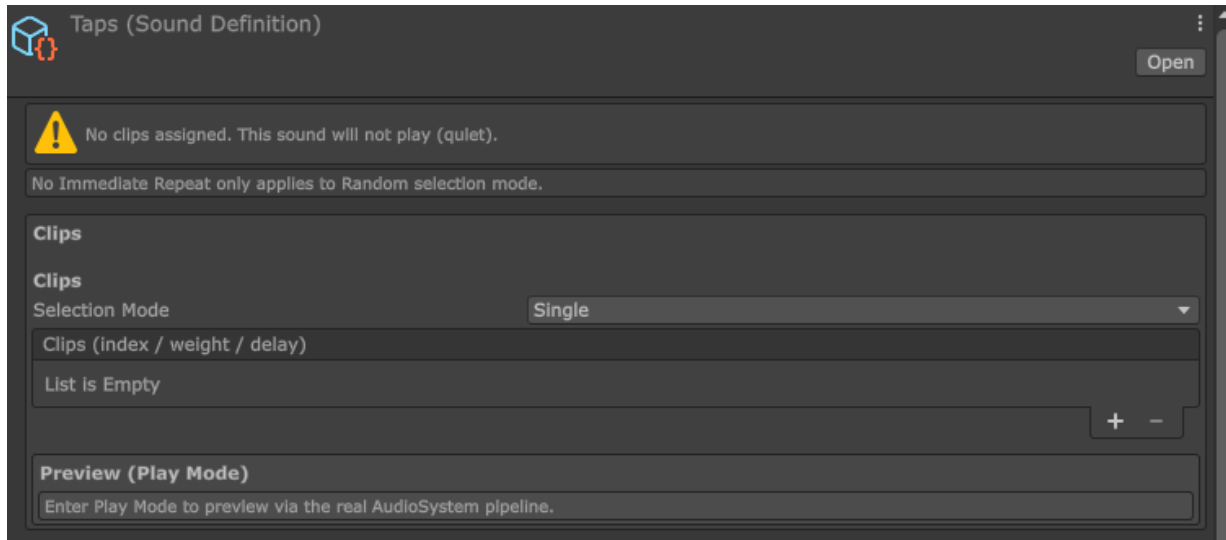
It is not built to impress.

It is built to prevent bugs.

Demo Scene Notice

The demo scene is included in this package for workflow demonstration purposes. Audio clips and music used in promotional videos or original demo presentations are not included in the package.

To test the system, please assign your own audio clips to the relevant demo objects or sound definitions.



- To test the system, please assign your own audio clips to the relevant demo objects or sound definitions.